Agent and tool use design

Chop 1 Quiz

You agent's complete tool architecture

In this activity, you will build the core intelligence for a research assistant. This is the key thing to get right at the start: you have learned how to write function calling, the ability of the AI to know the right tool at the right time.

Your primary goal is to initialize and define a custom function and integrate that assembly with both A client library (Google's Gemini SDK, Context) to enable complex, multi-step user requests within a conversational loop.

Scenario

You are tasked with building a prototype for an Al research assistant. This assistant must be able to search the web for recent news, save important notes to a file, and summarize information for the research team, operating smoothly within a single conversation.

Stages

Step 1: Setup and secure the agent

We begin by setting up the secure project environment and the base code that handles client initialization.

1. **Create project and secure Key:** Create a new project folder. Create a `.env` file for your API key and a `.gitignore` file to exclude `.env` (security best practice).
2. **Install library:** Install the necessary packages.
   - **Python (e.g., `requirements.txt`):** `google-genai`, `python-dotenv`
3. **Write starter code:** Copy the following code into your main file.

(Python starter code)

```python
# agent.py
import os
from typing import Any, Dict, List
from dotenv import load_dotenv
from google import genai
from google.genai import types

# --- Setup & Client ---
load_dotenv()
API_KEY = os.getenv("GEMINI_API_KEY")
if not API_KEY:
    raise ValueError("GEMINI_API_KEY is not set.")

client = genai.Client(api_key=API_KEY)
MODEL_ID = "gemini-2.5-flash-preview" # Using a capable model
```

1. You're sifting through the project for your Al research assistant. You know your Gemini API key is stored in `.env` file and don't want to hardcode it. How do you load this key into your Python script without exposing it in the code, so you can safely share your project on GitHub. Which of the following is the most secure and standard practice? Select the best answer. 1 point

   - Create a separate Python file named `secrets.py` , store the key in a variable there, and import it into your main script.
   - Paste the API key directly into your main script as a string variable.
   - Create a `.env` file in the root of the project, add one line: `GEMINI_API_KEY=your_actual_key`, and use a library to load the key as an environment variable in your script.
   - Encrypt the key and include the encrypted string and the decryption password in the code.

Step 2: Implement in-context functions and schema

You must implement the definitions for the three functions the agent will use.

Action:

1. **Analyze the `save_note` function:** Examine the provided Python function with error handling provided as an example.
2. **Analyze the schema:** Review the structured file `save_note_schema`. This is given as a starting point.

Define tools in a Schema for the other two: key-value pairs from `lookup_context` (`generate_content_schema`)

```python
# --- 2. Tool Definitions ---
def save_note(content: str, filename: str) -> Dict[str, Any]:
    """Saves the provided content to a specified file."""
    try:
        with open(filename, "w", encoding="utf-8") as f:
            f.write(content + "\n")
        return {"status": "success", "message": f"Note saved to {filename}."}
    except Exception as e:
        return {"status": "error", "message": f"Failed to save note: {e}"}

def web_research(query: str, url: str = None) -> Dict[str, Any]:
    """
    Performs web research. Can retrieve the page from a URL or summarize an
    this is where an integration like google_search or url_context tools
    """
    if query:
        if "google_search" in:
            print("Google Search is not implemented in this example.")
        if url:
            print(f"URL context is not implemented in this example.")

    print(f"Performing research for query: '{query}'")
    # Mock response
    return {"status": "success", "message": f"Research complete. Found relevant information about '{query}'."}

# Map function names to their implementations
available_functions = {
    "save_note": save_note,
    "web_research": web_research,
}

# --- Tool Schema for the Model ---
tools = [
    {
        "function_declarations": [
            {
                "name": "save_note",
                "description": "Saves important information or a summary to a specified file. Use this to remember findings.",
                "parameters": {
                    "type": "OBJECT",
                    "properties": {
                        "content": {"type": "STRING", "description": "The text content to save."},
                        "filename": {"type": "STRING", "description": "The name of the file, e.g., 'notes.txt'."},
                    },
                    "required": ["content", "filename"],
                },
            },
            {
                "name": "web_research",
                "description": "Performs research to find recent, relevant information about a topic.",
                "parameters": {
                    "type": "OBJECT",
                    "properties": {
                        "query": {"type": "STRING", "description": "The search query or topic to research."},
                        "url": {"type": "STRING", "description": "An optional URL to get context from."},
                    },
                    "required": ["query"],
                },
            }
        ]
    }
]
```

1. **You have defined a function `save_note(content: str, filename: str)` in your Colaport. Your provide a prompt: "Please save the key takeaways from our meeting into a file called meeting_notes.txt" However, the model does not call your function and instead just replies with text. What is the most likely reason the function was not called? Select the best answer.** 1 point

   - The model is not capable of writing to files, so it ignored the function.
   - The function's name, its docstring (`"""Saves..."""`), and its schema work with the model.
   - The function's description in the schema was unclear or did not accurately explain that it saves content to a file.
   - You forgot to include `save_note` as a string in your `available_functions` dictionary.

2. **When defining the parameters in `save_note_schema`, for a valid function, you decide to mark which the filename is optional. What is the most important reason why explicitly listing parameters as required is a best practice?** 1 point

   - It allows the model to ignore the user's instructions if it thinks the filename is not important.
   - It automatically creates a new folder on the developer's computer to store the notes.
   - It helps the model to always identify and provide a value for the parameters (without attempting to call the function.
   - It makes the AI's Python code run significantly faster when opening the file.

Step 3: Action log the orchestration

This action handles any multi-function requests initiated by the model, runs the appropriate local logic with the provided inputs, and produces a structured result. That result is then packaged into the correct format so it can be added back into the conversation history. Because a multi-turn response, it also includes a unique ID to consistently connect each function call with its corresponding result.

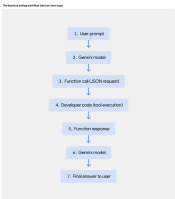
Action 1: Define all tools. Finish the code below with the missing text to create a single tool configuration object that includes your three custom functions and the web search function tool.

```python
# --- 2. Tool Definitions ---
def run_code_tool(code: str) -> Dict[str, Any]:
    """Executes a block of Python code and returns its output."""
    print(f"Executing code:\n---\n{code}\n---")
    try:
        # Using exec() is powerful but dangerous. Only run code you trust.
        exec(code, {'__builtins__': __builtins__}, {})
        return {"status": "success", "output": "Code executed successfully."}
    except Exception as e:
        return {"status": "error", "output": f"Error executing code: {e}"}

available_functions = {
    "save_note": save_note,
    "web_research": web_research,
    "run_code_tool": run_code_tool,
}

tools = [
    {
        "function_declarations": [
            {
                "name": "save_note",
                "description": "Saves important information or a summary to a specified file. Use this to remember findings.",
                "parameters": {
                    "type": "OBJECT",
                    "properties": {
                        "content": {"type": "STRING", "description": "The text content to save."},
                        "filename": {"type": "STRING", "description": "The name of the file, e.g., 'notes.txt'."},
                    },
                    "required": ["content", "filename"],
                },
            },
            {
                "name": "web_research",
                "description": "Performs research to find recent, relevant information about a topic.",
                "parameters": {
                    "type": "OBJECT",
                    "properties": {
                        "query": {"type": "STRING", "description": "The search query or topic to research."},
                        "url": {"type": "STRING", "description": "An optional URL to get context from."},
                    },
                    "required": ["query"],
                },
            },
            {
                "name": "run_code_tool",
                "description": "Executes a block of Python code and returns its output. Use this for calculations or data processing.",
                "parameters": {
                    "type": "OBJECT",
                    "properties": {
                        "code": {"type": "STRING", "description": "The Python code to execute."},
                    },
                    "required": ["code"],
                },
            }
        ]
    }
]
```

The function calling workflow (Review last task)



Action 2: Implement the main turn-by-turn conversation loop: Implement the overall conversation loop that handles function calls (the model's requests to use your Python code) and then sends the results back to the model for a final, user-friendly response.

- **The manager-employee analogy:** The function calling mechanism works in two turns. The first time manager asks your employee (the model) to perform a task. The employee uses a tool and sends the results back (turn 2). The process the follows. Having all the work (in the second turn (turn 3).
- **The OPTIMIZE ZONE MODE:** Also code in the "assemble all" of the function calling results (like the model's requested function calls), run them in Python, and package the results back into the chat history so the model can see them.
- **Run the script** and invoke a task that only testing the different tools.

```python
# --- 3. Function Handling and Execution ---
def handle_tool_call(history: List[types.Content], user_prompt: str) -> str:
    """Handles the multi-turn tool calling conversation."""
    history.append(
        types.Content(
            role="user", parts=[types.Part(text=user_prompt)],
        )
    )

    print("\nSending request to model...")
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=history,
        config=types.GenerateContentConfig(
            tools=tools,
        ),
    )

    # Check if the model wants to call a function
    if response.candidates[0].content.parts[0].function_call:
        function_call = response.candidates[0].content.parts[0].function_call
        function_name = function_call.name
        function_args = {key: value for key, value in function_call.args.items()}

        print(f"Model wants to call function: {function_name} with args: {function_args}")

        # Execute the function
        if function_name in available_functions:
            function_to_call = available_functions[function_name]
            function_response = function_to_call(**function_args)

            # Send the function's result back to the model
            history.append(response.candidates[0].content) # Add model's request
            history.append(
                types.Content(
                    role="tool",
                    parts=[
                        types.Part.from_function_response(
                            name=function_name,
                            response=function_response,
                        )
                    ],
                )
            )

            print("Sending function response back to model...")
            final_response = client.models.generate_content(
                model=MODEL_ID,
                contents=history,
                config=types.GenerateContentConfig(
                    tools=tools,
                ),
            )
            return final_response.text
        else:
            return f"Error: Model tried to call unknown function '{function_name}'"
    else:
        # Model responded directly
        return response.text

# --- 4. Main Execution Block ---
if __name__ == "__main__":
    print("Research Assistant Initialized. Type 'exit' to quit.")
    chat_history = []
    while True:
        user_input = input("\nYou: ")
        if user_input.lower() == 'exit':
            print("Goodbye!")
            break

        final_answer = handle_tool_call(chat_history, user_input)
        print(f"\nAssistant: {final_answer}")
```

1. **Your Al agent has received a function call from Gemini for its `run_code_tool`. In your `run_tool_call` calling loop, you execute the function for this.**

   Code:

   ```
   result = run_code_tool(code_to_run)
   ```

   To complete the tool turn and finish it. We continue or Gemini can use the function output when generating the final response, what should you do next? Select the best answer.

   - Append a JSON to give the function, `function_response(...)` append it as type `Content` under "tool" parse (...) to history, then call generate_content() again.
   - Call client.models.generate_content() again using the same history without appending anything new.
   - Print result and end the turn, since the output is already for the function.
   - Replace the tool result with a natural-language summary and add it as a new "user" message.

2. **In a research assistant application, the agent determines it needs to call a function. The developer's code executes the local Python logic, and gets a result. Why must this result be appended back to the history as a "tool" role before calling the model's `generate_content` below the last answer.**

   - To change the model's personality to be more professional and concise
   - To verify the user that the code has finished running so they can close the terminal
   - To provide the model with the missing information it requested, allowing it to complete the user's original task
   - To save the user's API prompt and store it as a history conversation from us with

Final checklist

Your Al agent is architecture is complete when:

1. The three custom functions (save_note, web_research, run_code_tool) and schema declarations.
2. The **`available_functions`** list correctly includes your three custom functions plus the Google Search and URL Context tools.
3. The tool turn and iteration loop correctly handles the function call request, executes the function, and sends the result back to the model.

Next steps

The next step is to test your agent by providing a user prompt that requires it to use all three tools sequentially. Your agent should proceed by chaining these tools together to achieve the final output.

Key points

- **Function declarations:** The most critical step: A function calling is setting a clear, accurate description (the **docstring** you made. You are the schema. The AI relies entirely on this text to decide whether or not to call your code.
- **Agent can be helpful:** You do not need to manually code the sequence! Gemini or Summarize' re Client. By providing the tools and the history of the tools and the function correct decisions, the intelligence handles the complex, multi-step orchestration automatically.

Example solution (Explored)

(The Gemini for shown the required first) Type, for the definitions and the execution map.)

```python
# agent.py
import os
from typing import Any, Dict, List
from dotenv import load_dotenv
from google import genai
from google.genai import types

# --- Setup & Client ---
load_dotenv()
API_KEY = os.getenv("GEMINI_API_KEY")
if not API_KEY:
    raise ValueError("GEMINI_API_KEY is not set.")

client = genai.Client(api_key=API_KEY)
MODEL_ID = "gemini-2.5-flash-preview" # Using a capable model

# --- 2. Tool Definitions (Summary Agent) ---
def save_note(content: str, filename: str) -> Dict[str, Any]:
    """Saves a note to a local text file."""
    try:
        with open(filename, "w", encoding="utf-8") as f:
            f.write(content + "\n")
        return {"status": "success", "message": f"Note saved to {filename}."}
    except Exception as e:
        return {"status": "error", "message": f"Failed to save note: {e}"}

def analyze_sentiment(text: str) -> Dict[str, Any]:
    """Analyzes the sentiment of a piece of text."""
    print(f"Analyzing sentiment for: '{text}'")
    # Mock response
    if "happy" in text or "great" in text or "excellent" in text:
        sentiment = "positive"
    elif "sad" in text or "bad" in text or "terrible" in text:
        sentiment = "negative"
    else:
        sentiment = "neutral"
    return {"status": "success", "sentiment": sentiment}

# Map function names to their implementations
available_functions = {
    "save_note": save_note,
    "analyze_sentiment": analyze_sentiment,
}

# --- Tool Schema for the Model ---
tools = [
    {
        "function_declarations": [
            {
                "name": "save_note",
                "description": "Saves important information or a summary to a specified file. Use this to remember findings.",
                "parameters": {
                    "type": "OBJECT",
                    "properties": {
                        "content": {"type": "STRING", "description": "The text content to save."},
                        "filename": {"type": "STRING", "description": "The name of the file, e.g., 'notes.txt'."},
                    },
                    "required": ["content", "filename"],
                },
            },
            {
                "name": "analyze_sentiment",
                "description": "Analyzes the sentiment (positive, negative, or neutral) of a piece of text.",
                "parameters": {
                    "type": "OBJECT",
                    "properties": {
                        "text": {"type": "STRING", "description": "The text to analyze."},
                    },
                    "required": ["text"],
                },
            }
        ]
    }
]

# --- 3. Function Handling and Execution ---
def handle_tool_call(history: List[types.Content], user_prompt: str) -> str:
    """Handles the multi-turn tool calling conversation."""
    history.append(
        types.Content(
            role="user", parts=[types.Part(text=user_prompt)],
        )
    )

    print("\nSending request to model...")
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=history,
        config=types.GenerateContentConfig(
            tools=tools,
        ),
    )

    # Check if the model wants to call a function
    if response.candidates[0].content.parts[0].function_call:
        function_call = response.candidates[0].content.parts[0].function_call
        function_name = function_call.name
        function_args = {key: value for key, value in function_call.args.items()}

        print(f"Model wants to call function: {function_name} with args: {function_args}")

        # Execute the function
        if function_name in available_functions:
            function_to_call = available_functions[function_name]
            function_response = function_to_call(**function_args)

            # Send the function's result back to the model
            history.append(response.candidates[0].content) # Add model's request
            history.append(
                types.Content(
                    role="tool",
                    parts=[
                        types.Part.from_function_response(
                            name=function_name,
                            response=function_response,
                        )
                    ],
                )
            )

            print("Sending function response back to model...")
            final_response = client.models.generate_content(
                model=MODEL_ID,
                contents=history,
                config=types.GenerateContentConfig(
                    tools=tools,
                ),
            )
            return final_response.text
        else:
            return f"Error: Model tried to call unknown function '{function_name}'"
    else:
        # Model responded directly
        return response.text

# --- 4. Main Execution Block ---
if __name__ == "__main__":
    print("Research Assistant Initialized. Type 'exit' to quit.")
    chat_history = []
    while True:
        user_input = input("\nYou: ")
        if user_input.lower() == 'exit':
            print("Goodbye!")
            break

        final_answer = handle_tool_call(chat_history, user_input)
        print(f"\nAssistant: {final_answer}")
```

How does your work compare?

Use these questions to evaluate the quality and robustness of your final agent architecture:

- **Function clarity:** Will you write a clear, descriptive explanation of where descriptions for all three custom functions? The model relies entirely on this text to decide when to call your code.
- **Complete toolset:** Did you successfully configure the API to enable access to all the required tools (your three custom functions + Google Search + URL Context)?
- **Robust logic:** Does your code correctly implement the multi-turn conversation loop correctly implement the function process? Is it...Did you check for the model's request, execute the function, and then send the result back to the model for the final response?
- **Robustness:** Did you add a try/except block around your code to prevent a crash if the user provides a bad input (e.g., a non-existent file)? Does it return a helpful error message to the model? Or the operation fails?
- **Local error handling:** Did you wrap any local file and storage (like the reading/writing a file) — **even if Gemini** should return a friendly error message to the model? Or the operation fails?